



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 16

по курсу «Языки и методы программирования»

«N-body problem. BigInteger. Рациональная арифметика. ncurses»

Студент группы ИУ9-22Б
Булдаков А. С.

Преподаватель
Посевин Д. П.

Содержание

Цель работы	1
Архитектура проекта	1
BigInteger и Rational	1
BigInteger	1
Rational	2
Класс Body	3
Класс Universe	4
Класс Visualizer	5
Возможности	5
Сценарий threeBody	6
Сборка и запуск	7
Тестирование	8
Вывод	8

Цель работы

Реализовать симуляцию гравитационного взаимодействия N тел с использованием арифметики произвольной точности (BigInteger и Rational). Визуализировать движение тел в консоли с помощью библиотеки ncurses.

Архитектура проекта

Проект состоит из пяти основных компонентов:

- BigInteger / Rational — библиотека арифметики произвольной точности;
- Body — представление одного небесного тела;
- Universe — физический движок (расчёт сил, интеграция уравнений движения);
- Visualizer — консольная визуализация через ncurses;
- test.cpp — модульные тесты.

Сборка осуществляется через CMake с единственной внешней зависимостью — ncursesw.

BigInteger и Rational

Ядро проекта — реализация целых чисел произвольной точности (BigInteger) и рациональных чисел (Rational) на их основе.

BigInteger

BigInteger хранит число в системе с основанием 2^{32} с помощью `std::vector<unsigned int>`. Поддерживаются:

- сложение, вычитание, умножение, деление, остаток;
- бинарные сдвиги (`<<`, `>>`);
- конвертация в `double` и строку произвольной точности;
- пользовательский литерал `_bi`.

Умножение реализовано бинарным алгоритмом (сдвиг + сложение), деление — побитовым делением в столбик.

include/biginteger.h — фрагмент

```
class BigInteger {
    std::vector<unsigned int> digits;
    bool isNegative;

public:
    BigInteger() : isNegative(false) { digits.push_back(0); }
    BigInteger(long long num);

    BigInteger operator+(const BigInteger &other) const;
    BigInteger operator-(const BigInteger &other) const;
    BigInteger operator*(const BigInteger &other) const;
    BigInteger operator/(const BigInteger &other) const;

    explicit operator double() const;
    std::string toString() const;

    friend BigInteger operator""_bi(const char *str);
};
```

Rational

Rational представляет число в виде числитель / знаменатель (оба — BigInteger). После каждой операции выполняется нормализация через GCD. Поддерживаются:

- конструкторы от int, BigInteger, double, const char* (включая экспоненциальную запись);
- арифметические операции;
- метод asDecimal(precision) для форматированного вывода;
- конвертация в double.

include/biginteger.h — фрагмент Rational

```
class Rational {
    BigInteger num;
    BigInteger den;

    void normalize();

public:
    Rational() : num(0), den(1) {}
    Rational(BigInteger n, BigInteger d);
    Rational(double d);
    Rational(const char *decimalStr);

    Rational operator+(const Rational &other) const;
    Rational operator-(const Rational &other) const;
    Rational operator*(const Rational &other) const;
    Rational operator/(const Rational &other) const;

    std::string asDecimal(int precision) const;
    explicit operator double() const;
};
```

Класс Body

Хранит состояние одного тела: координаты (x, y), скорость (vx, vy) и массу. Все величины имеют тип Rational.

include/Body.hpp

```
class Body {
    Rational m_x, m_y;
    Rational m_vx, m_vy;
    Rational m_mass;

public:
    Body(const Rational &x, const Rational &y,
         const Rational &vx, const Rational &vy,
         const Rational &mass);

    Rational getX() const;
    Rational getY() const;
    Rational getVx() const;
    Rational getVy() const;
    Rational getMass() const;

    void setX(const Rational &x);
    void setY(const Rational &y);
    void setVx(const Rational &vx);
    void setVy(const Rational &vy);
};
```

Класс Universe

Содержит вектор тел и реализует шаг симуляции гибридным методом:

1. Расчёт ускорений выполняется в `double` (требуется `std::sqrt`):
 - для каждой пары (i, j) вычисляется расстояние $R = \sqrt{dx^2 + dy^2}$;
 - ускорение $a_j += G m_i / R^3 (r_i - r_j)$.
2. Обновление позиций и скоростей выполняется в `Rational`:
 - $x += v * dt + 0.5 * a * dt * dt$ (Taylor expansion второго порядка);
 - $v += a * dt$.

Сложность алгоритма — $O(N^2)$ (direct summation, без Barnes-Hut). Параллелизация отсутствует.

```

void Universe::step(const Rational &dt) {
    size_t n = m_bodies.size();
    std::vector<double> ax(n, 0.0), ay(n, 0.0);
    double G = m_G.toDouble();

    // Вычисление ускорений (двойная точность для sqrt)
    for (size_t j = 0; j < n; ++j) {
        double xj = m_bodies[j].getX().toDouble();
        double yj = m_bodies[j].getY().toDouble();
        for (size_t i = 0; i < n; ++i) {
            if (i == j) continue;
            double xi = m_bodies[i].getX().toDouble();
            double yi = m_bodies[i].getY().toDouble();
            double mi = m_bodies[i].getMass().toDouble();
            double dx = xi - xj;
            double dy = yi - yj;
            double R = std::sqrt(dx * dx + dy * dy);
            double R3 = R * R * R;
            ax[j] += G * mi / R3 * dx;
            ay[j] += G * mi / R3 * dy;
        }
    }

    // Обновление состояния (рациональная арифметика)
    for (size_t i = 0; i < n; ++i) {
        Rational newX = m_bodies[i].getX()
            + m_bodies[i].getVx() * dt
            + Rational(ax[i]) * dt * dt / Rational(2);
        Rational newY = m_bodies[i].getY()
            + m_bodies[i].getVy() * dt
            + Rational(ay[i]) * dt * dt / Rational(2);
        Rational newVx = m_bodies[i].getVx() + Rational(ax[i]) * dt;
        Rational newVy = m_bodies[i].getVy() + Rational(ay[i]) * dt;

        m_bodies[i].setX(newX);
        m_bodies[i].setY(newY);
        m_bodies[i].setVx(newVx);
        m_bodies[i].setVy(newVy);
    }
}

```

Класс Visualizer

Использует библиотеку ncursesw для консольной анимации.

Возможности

- отображение тел символами 'O' разных цветов (до 7 цветовых пар);
- автоматический расчёт масштаба по начальным координатам;

- HUD: номер шага, dt, количество тел, состояние паузы;
- управление: q (выход), p (пауза), + (ускорить в 1.5 раза), - (замедлить в 1.5 раза).

include/Visualizer.hpp

```
class Visualizer {
    Universe &m_universe;
    Rational m_dt;
    bool m_paused;
    bool m_running;
    double m_viewX, m_viewY, m_scale;

public:
    Visualizer(Universe &u, const Rational &dt);

    void init();
    void run();

    void setDt(const Rational &dt);

private:
    void render();
    void handleInput();
    void shutdown();
};
```

Сценарий threeBody

В main.cpp реализован демонстрационный сценарий — задача трёх тел с нормализованными параметрами:

src/main.cpp

```
int main() {
    Universe universe(Rational("1")); // G = 1

    universe.addBody(Body(-10, 0, 0, -2, 100));
    universe.addBody(Body(10, 0, 0, 2, 100));
    universe.addBody(Body(0, 10, 1.5, 0, 100));

    Rational dt = Rational(2) / Rational(100);
    Visualizer visualizer(universe, dt);
    visualizer.init();
    visualizer.run();

    return 0;
}
```

Три тела равной массы (100) расположены в вершинах треугольника и движутся по сложной траектории, демонстрируя хаотическое поведение, характерное для задачи трёх тел.

Сборка и запуск

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)
project(nbody LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

find_package(PkgConfig REQUIRED)
pkg_check_modules(NCURSES REQUIRED ncursesw)

add_executable(nbody
    src/main.cpp src/Body.cpp src/Universe.cpp
    src/Visualizer.cpp src/biginteger.cpp
)
target_include_directories(nbody PRIVATE include
    ${NCURSES_INCLUDE_DIRS})
target_link_libraries(nbody PRIVATE ${NCURSES_LIBRARIES})
target_compile_options(nbody PRIVATE -Wall -Wextra -Wpedantic)

add_executable(nbody_test
    src/test.cpp src/Body.cpp src/Universe.cpp src/biginteger.cpp
)
target_include_directories(nbody_test PRIVATE include)
target_compile_options(nbody_test PRIVATE -Wall -Wextra -Wpedantic)
```

Makefile

```
build:
    cmake -B build -DCMAKE_BUILD_TYPE=Release
    cmake --build build

run: build
    ./build/nbody

test: build
    ./build/nbody_test

clean:
    rm -rf build

.PHONY: build run test clean
```

Запуск: `make run` (сборка + запуск) или `make test` (модульные тесты).

Тестирование

Тесты реализованы в `src/test.cpp` с использованием простого макросного фреймворка (`TEST / END_TEST`, проверка через `assert`):

- конструкторы `Rational` от строки и `double`;
- арифметические операции с большими числами;
- конвертация `Rational` $\rightarrow$ `double` $\rightarrow$ `Rational` (round-trip);
- тест гравитационной постоянной `G`;
- конструктор и геттеры/сеттеры `Body`;
- симуляция `Universe::step()` на 100 и 2000 шагов (проверка отсутствия NaN/Inf).

Вывод

В ходе лабораторной работы реализована симуляция гравитационного взаимодействия N тел с использованием:

- арифметики произвольной точности (`BigInteger` с основанием 2^{32} , `Rational` с GCD-нормализацией);
- прямого $O(N^2)$ алгоритма расчёта сил с гибридной точностью (`double` для ускорений, `Rational` для координат);
- консольной визуализации через `ncursesw` с цветами и управлением в реальном времени.

Код отформатирован по стилю LLVM (`.clang-format`) и проверен статическим анализатором (`.clang-tidy`). Сборка осуществляется через CMake с единственной зависимостью `ncursesw`.